# NASA Kepler Data + AstroNet-style 1D CNN

Machine learning pipeline for classifying stellar transit signals as confirmed planets or false positives

**Team:** Francisco Medina| **Tier:** 2

Made with GAMMA

# The Problem

## What Real-World Problem

Astronomers need to distinguish genuine exoplanet transit signals from false positives in Kepler light curve data

## Who Cares

Astrophysicists, NASA researchers, and exoplanet discovery teams analyzing Kepler mission data

## Why Important

Manual classification is time-consuming and subjective; automated systems enable faster discovery of potentially habitable worlds

Made with GAMMA

# My Solution

Automated exoplanet detection system using deep learning to classify Kepler light curves

1

## Input

Kepler star ID, orbital period, transit epoch

2

## Processing

1D CNN analyzes phase-folded light curves

3

## Output

Planet detection probability (0-100%)

Made with GAMMA

# Technical Approach

## CV Technique

Binary classification of time series data

## Model

1D Convolutional Neural Network (AstroNet-style architecture)

## Framework

TensorFlow/Keras with scikit-learn for evaluation

## Why This Approach

1D CNNs excel at detecting patterns in sequential data like light curves. The architecture mimics proven AstroNet designs that achieved high accuracy on NASA exoplanet data.

Made with GAMMA

# Data Plan

## Source

NASA Exoplanet Archive — Kepler Cumulative KOI Catalog (public dataset)

## Size

Up to 400 light curves attempted (200 confirmed + 200 false positives) — actual usable count depends on quality filtering

## Labels

Pre-labeled by NASA: CONFIRMED (class 1) vs FALSE POSITIVE (class 0)

## Quality Filters

4-stage pipeline: download success, NaN rate <10%, signal variance check, data sanitization

Made with GAMMA

# System Diagram

01

## Download KOI Catalog

Fetch labeled dataset from NASA archive

02

## Retrieve Light Curves

Use lightkurve to download Kepler data

03

## Preprocess Data

Flatten, fold, bin, normalize flux arrays

04

## Train 1D CNN

Convolutional blocks + dense classifier

05

## Evaluate Model

Confusion matrix, ROC, precision-recall curves

06

## Predict New Signals

Classify any Kepler star by parameters

Made with GAMMA

# Success Metrics

**59.38%**

**Target Accuracy**

Test Accuracy — Final model performance

**0.573**

**ROC Performance**

Test AUC — ROC curve performance

**7.4ms**

**Processing Time**

Processing Time — Per light curve prediction

**Primary Metric:** Test accuracy (59.38%) and AUC (0.573)

**Secondary Metrics:** Precision (59.57%), Recall (80.00%), processing time per prediction

Target vs Actual: Accuracy target was >80%, actual test accuracy was 59.38%. AUC target was >0.85, actual test AUC was 0.573. The model achieved strong recall (80%) but lower precision (59.57%), indicating it correctly identifies most planets but has false positives.

Made with GAMMA

# Week-by-Week Plan

1

### Week 1 (Feb 17)

Get dataset, set up environment, install dependencies

2

### Week 2 (Feb 24)

Train/fine-tune 1D CNN model architecture

3

### Week 3 (Mar 3)

Test, evaluate results, and complete documentation

4

### Completed (Mar 6)

Project finished! 🎉

Made with GAMMA

# Challenges & Backup Plans

### Challenge: Data Quality Issues

**Problem:** Light curves with too many NaN values or poor signal-to-noise ratio

**Plan B:** Increase sample size from NASA catalog, use more aggressive filtering, or apply data imputation techniques

### Challenge: Low Accuracy

**Problem:** Model fails to achieve target performance metrics

**Plan B:** Try deeper CNN architecture, add dropout layers, use data augmentation, or experiment with different hyperparameters

### Challenge: Download Failures

**Problem:** NASA MAST archive unavailable or rate limiting

**Plan B:** Use cached dataset from previous runs, reduce sample size, or switch to alternative data sources

Made with GAMMA

# Implementation Details

## Compute Environment

- Google Colab (T4 GPU recommended)
- TensorFlow 2.x + Keras
- lightkurve library for data access
- scikit-learn for evaluation

## Model Architecture

- 4 convolutional blocks (16-32-64-128 filters)
- MaxPooling between blocks
- Dense head with dropout
- Sigmoid output for binary classification
- Train/Val/Test split: 68% / 12% / 20%

## Training Configuration

- Adam optimizer (1e-4 learning rate)
- Binary crossentropy loss
- 60 epochs with early stopping
- Batch size: 32
- Input shape: (201, 1)

## Cost

**Free:** Google Colab runtime with GPU support

Made with GAMMA